

**Theory Assignment -1 (Unit 1 to 4, 7,10)**

**Unit 1: Introduction to Java & Program Structure**

1. Explain the **Java execution process** starting from source code compilation to bytecode execution by the JVM.

2. Define the following terms and explain their role in the Java ecosystem:

a) JVM

b) JRE

c) JDK

d) Bytecode

3. Discuss the **key features of Java** that make it platform independent and suitable for distributed applications.

4. Explain the significance of **Unicode support in Java** and how it differs from ASCII-based programming languages.

5. Write a Java program that reads **three command-line arguments and prints them in reverse order**.

6. Write a program demonstrating the use of **System.out.printf()** for formatted output.

7. Predict the output of the following program:

```
class Test {  
    public static void main(String[] args) {  
        System.out.println(10 + 20 + "Java");  
        System.out.println("Java" + 10 + 20);  
    }  
}
```

8. Identify the error in the following program and correct it:

```
class Test {  
    public static void main(String args) {  
        System.out.println(args.length);  
    }  
}
```

9. Predict the output:

```
class Test {  
    public static void main(String[] args) {  
        System.out.println('A' + 'B');  
    }  
}
```

10. Evaluate the advantages of **Java bytecode architecture compared to native compiled languages like C++**.

11. Discuss the importance of **Java Virtual Machine architecture in ensuring security and portability**.

12. Design a Java program that simulates a **simple command-line calculator** using **command-line arguments**.

---

**Unit 2: Data Types, Variables, Operators & Loops**

1. List all **primitive data types in Java** and explain their memory size and range.
2. Define **operator precedence and associativity** with examples.
3. Explain the difference between **primitive data types and reference data types** with examples.
4. Explain the working of **bitwise operators in Java**.
5. Write a program to find the **largest among three numbers using conditional operators**.
6. Write a Java program to generate the **first 10 Fibonacci numbers using loops**.
7. Predict the output:

```
class Test {  
    public static void main(String[] args) {  
        int x = 5;  
        int y = x++ + ++x;  
        System.out.println(y);  
    }  
}
```

8. Predict the output:

```
class Test {  
    public static void main(String[] args) {  
        for(int i=0;i<3;i++){  
            {  
                System.out.println("Java");  
            }  
        }  
    }  
}
```

9. Identify the logical error in the following code:

```
int i = 1;  
while(i < 10);  
{  
    System.out.println(i);  
    i++;  
}
```

10. Evaluate the advantages and limitations of using **switch statements compared to if-else ladders**.
  11. Discuss how **type promotion affects mixed arithmetic expressions in Java**.
  12. Design a Java program to **generate prime numbers within a given range using loops and conditional statements**.
- 

**Unit 3: Classes, Objects & Constructors**

**Java Programming (Subject Code: MCA 124)**

1. Define **class and object** in Java and explain their relationship.
2. Explain the purpose of **constructors in object-oriented programming**.
3. Explain the difference between **instance variables and static variables**.
4. Explain **method overloading and constructor overloading with examples**.
5. Write a Java program to create a **Student class with attributes and display method**.
6. Write a program demonstrating **constructor overloading**.
7. Predict the output:

```
class Test {
```

```
    static{  
        System.out.println("Static");  
    }
```

```
    {  
        System.out.println("Instance");  
    }
```

```
    Test(){  
        System.out.println("Constructor");  
    }
```

```
    public static void main(String args[]){  
        new Test();  
    }  
}
```

8. Predict the output:

```
class Test {
```

```
    Test(){  
        this(10);  
        System.out.println("Default");  
    }
```

```
    Test(int a){  
        System.out.println("Parameterized");  
    }
```

```
    public static void main(String args[]){  
        new Test();  
    }  
}
```

9. Predict the output:

```
class Test {  
  
    static int x = 10;  
  
    public static void main(String[] args) {  
        Test t1 = new Test();  
        Test t2 = new Test();  
  
        t1.x = 20;  
  
        System.out.println(t2.x);  
    }  
}
```

10. Evaluate the importance of **encapsulation in class design**.

11. Discuss the role of **initializer blocks and static blocks during object creation**.

12. Design a **BankAccount** class with constructors, deposit, withdrawal, and balance display methods.

---

#### Unit 4: Inheritance, Interfaces & Lambda Expressions

1. Define **inheritance** and explain the “is-a” relationship.

2. What is an **interface** in Java?

3. Explain the difference between **abstract classes and interfaces**.

4. Explain the concept of **functional interfaces and lambda expressions**.

5. Write a program demonstrating **single inheritance**.

6. Write a Java program implementing an **interface with a default method**.

7. Predict the output:

```
class A{  
    void show(){  
        System.out.println("A");  
    }  
}
```

```
class B extends A{  
    void show(){  
        System.out.println("B");  
    }  
}
```

```
class Test{  
    public static void main(String[] args){  
        A obj = new B();  
        obj.show();  
    }  
}
```

```
}  
}
```

8. Predict the output:

```
interface A{  
    default void show(){  
        System.out.println("Interface");  
    }  
}  
  
class Test implements A{  
    public static void main(String[] args){  
        new Test().show();  
    }  
}
```

9. Identify the error:

```
interface A{  
    void show();  
}
```

```
class Test implements A{  
}
```

10. Evaluate the advantages of **lambda expressions** compared to **anonymous classes**.

11. Discuss how **interfaces** enable **multiple inheritance** in Java.

12. Design a **calculator application** using a **functional interface** and **lambda expressions**.

---

### Unit 7: Exception Handling

1. Define **exception handling** in Java.

2. Differentiate between **checked and unchecked exceptions**.

3. Explain the purpose of **try, catch, finally, throw, and throws** keywords.

4. Explain the **Java exception hierarchy**.

5. Write a Java program that handles **division by zero** using **try-catch**.

6. Write a program demonstrating **multiple catch blocks**.

7. Predict the output:

```
class Test{  
    public static void main(String[] args){  
  
        try{  
            int a = 10/0;  
        }  
  
        catch(Exception e){  
            System.out.println("Catch");  
        }  
    }  
}
```

```
}

    finally{
        System.out.println("Finally");
    }
}
}
```

8. Predict the output:

```
class Test{
    public static void main(String[] args){

        try{
            return;
        }

        finally{
            System.out.println("Finally");
        }
    }
}
```

9. Identify the error:

```
try{
    int a = 10/0;
}
catch(IOException e){
}
```

10. Evaluate the importance of **exception handling** in improving program reliability.

11. Discuss the role of **finally blocks** in resource management.

12. Design a **custom exception class** called **InvalidAgeException** and demonstrate its usage in a Java program.

---

## Unit 6: Java I/O Package

1. What is the **java.io** package?

2. Explain the purpose of the **File** class.

3. Explain the difference between **byte streams** and **character streams**.

4. Explain the working of **BufferedReader** and **FileWriter** classes.

5. Write a program to **create a file** and **write data** to it using **FileWriter**.

6. Write a program to **read data from a file** using **BufferedReader**.

7. Predict the output:

```
File f = new File("example.txt");
System.out.println(f.getName());
```

8. Predict the output:

**Department of Computer Science, Gujarat University**

**MCA- SEM-2**

**Java Programming (Subject Code: MCA 124)**

```
File f = new File("data.txt");
```

```
System.out.println(f.exists());
```

**9.** Identify the error:

```
FileWriter fw = new FileWriter("data.txt");
```

```
fw.write("Hello");
```

**10.** Evaluate the advantages of **buffered streams** compared to **unbuffered streams**.

**11.** Discuss the importance of **file handling** in persistent data storage.

**12.** Design a Java program using **RandomAccessFile** to read and modify specific records in a file.